Woxsen University

School of Technology

B.Tech - Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Disaster Relief Resource and Camp Management System

Course Code: 25TU03MJM0

Course Name: Database Management Systems

Semester: III

Section: CSE Panthers

Academic Year: 2026-27

Submitted by

M SHREEVENKAT SATHVIK

Register Number: 25WU0101168

Under the Guidance of

Dr Sahithi

Department of Computer Science and Engineering

October 2026

CERTIFICATE

This is to certify that the project titled "Disaster Relief Resource and Camp Management System" is a bonafide Project Based Learning work carried out by M SHREEVENKAT SATHVIK (Register No. 25WU0101168), B.Tech in Computer Science and Engineering, during Academic Year 2026-27 under my guidance.

The report presents the design and implementation of a MySQL database and a connected Python application for the educational Project 60 assignment.

Faculty Guide: Dr Sahithi

Guide Signature: _____

Date: _____ Place: _____

DECLARATION

I, M SHREEVENKAT SATHVIK (Register No. 25WU0101168), declare that the project titled "Disaster Relief Resource and Camp Management System" is my Project Based Learning submission for Database Management Systems. The work described here is based on the implementation, tests, and project records cited in this report.

The report distinguishes the fictional demonstration data from real disaster operations. External background sources are acknowledged in the references. I understand that the university will evaluate this submission under its academic integrity rules.

Candidate Signature: _____

Date: _____ Place: _____

ACKNOWLEDGEMENT

I thank Dr Sahithi for guidance on this Project Based Learning work. I also thank the School of Technology for providing the Database Management Systems project framework and staged review process.

The project helped me connect classroom topics—entity relationships, normalization, SQL constraints, joins, transactions, and reports—to a practical relief-camp workflow. I have used fictional records for the demonstration so that the database design and validation rules can be discussed without handling real personal or emergency data.

ABSTRACT

This report describes the design and implementation of a Disaster Relief Resource and Camp Management System as a Database Management Systems Project Based Learning assignment. Relief camps need a clear record of registered families, their assessed supply needs, donations, warehouse stock, volunteers, item distributions, and referrals. Separate paper registers can repeat the same information and make it difficult to know whether a request remains open or whether stock has already been issued. The project uses a relational MySQL database and a small Python application to connect these records in one workflow. The schema has sixteen tables and seven reporting views. Primary keys, foreign keys, unique rules, checks, and indexes protect individual records. Application transactions check rules that involve several rows, including remaining family need, available stock, the volunteer's camp assignment, and duplicate issue requests. A receipt or distribution and its matching stock movement are committed together. The design was examined for First, Second, and Third Normal Form; a repeated item identifier was removed from the distribution table because the related need already determines that item. The practical innovation is an explainable unmet-needs queue sorted by urgency and assessment time. It is intentionally simple enough for staff and examiners to inspect. On fictional seed data, issuing three water kits changed one family's remaining requirement from five to two and warehouse stock from twenty to seventeen. Eight MySQL integration tests passed, including a concurrent issue check and ledger reconciliation. The result demonstrates how basic relational design and transactions can make a small relief workflow easier to audit. The local demonstration is not presented as a production emergency system.

Keywords: DBMS, MySQL, normalization, disaster relief, camp management, stock ledger, Python.

TABLE OF CONTENTS

Section	Page
Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
List of Figures	7
List of Tables	8
1 Introduction	9
2 Problem Statement and Objectives	12
3 Literature Review and Background	14
4 Methodology and System Design	18
5 Implementation	24
6 Results and Discussion	30
7 Conclusion and Future Scope	33
References	34

LIST OF FIGURES

Figure	Description	Page
1	Three-layer system architecture	19
2	Core entity relationship groups	20
3	Donation and distribution data flow	22
4	Application overview and unmet-needs queue	28
5	Distribution form and history	29
6	Verified demo quantities	31

LIST OF TABLES

Table	Description	Page
1	Requirements and database evidence	13
2	Related systems and this project	17
3	Main entities and identifiers	20
4	Representative integrity rules	22
5	Module responsibilities	24
6	Seed data counts	27
7	Verified query output	30
8	Test summary	32

1 INTRODUCTION

1.1 Background

A relief camp serves people displaced by an emergency while supplies and volunteers move through several locations. The same family may have several members and several item needs. One warehouse may receive the same item from several donors, and a need may be fulfilled in more than one issue. These are repeated relationships, so they suit a relational database better than a single wide paper register. The Project 60 statement asks for a database that records camps, families, needs, inventory, distribution, volunteers, and related reports [1].

The core problem is informational rather than algorithmic. A coordinator needs answers to simple questions: How many people are in each camp? Which assessed needs remain open? How much stock is on hand? Which volunteer issued an item, and when? If these answers come from unrelated lists, the team must reconcile them manually. A structured schema lets each fact have one clear home and uses keys to connect related facts [2].

This report examines a small local demonstration built with MySQL and Python. It uses fictional data and a controlled workflow. The project does not assume live connectivity across disaster sites or claim that a classroom prototype is ready for emergency deployment.

A family and a person are separate records because one registered family may contain several people, while a supply need is assessed for the family as a unit. Likewise, a warehouse balance is separate from a donation because several receipts and issues can affect the same item. These distinctions are the starting point for the ER model and for the later SQL joins.

1.2 Motivation and relevance

The humanitarian context explains why the data model matters. The Sphere Handbook organizes minimum standards around essential areas such as water, food, shelter, and health [4]. Although this project does not implement those standards as allocation rules, it uses item categories and referrals to reflect common relief activities. FEMA's resource inventory guidance also emphasizes keeping counts current for deployment and audit [9]. The design therefore treats stock as an accountable quantity, not only as a display number.

The project focuses on an ordinary but important sequence: assess an item need, receive a donation into a warehouse, assign a volunteer, and issue a controlled quantity. If the same family is assessed twice for the same item and round, a unique key rejects the duplicate. If an issue exceeds remaining need or stock, the service rejects it. These rules show how database constraints and transactions support decisions without replacing human judgment.

The unmet-needs queue is the project's practical improvement over a plain data-entry system. It orders open needs by the declared urgency and then by assessment time. A coordinator can see why one row appears before another. The order is transparent and can be challenged or corrected by changing the assessment rather than interpreting a hidden score.

Urgency values are HIGH, MEDIUM, and LOW. An assessment time breaks ties within a level; the queue therefore has a predictable rule. The remaining quantity is calculated from the assessed amount and recorded distributions, so a partly served request stays visible. The queue is a report, not an allocation algorithm: staff still decide which item to issue and must use the controlled distribution form.

1.3 Scope and report organization

The implemented scope includes disaster and camp details, family and person registration, relief item catalogues, needs assessment, donors and receipts, warehouse stock, volunteers and assignments, distributions, referrals, and SQL reports. The database contains sixteen tables and seven views. The Python application offers forms and tables for the main workflows. The repository also contains schema scripts, fictional sample records, screenshots, review notes, and integration tests [18].

The local prototype assumes a trusted operator and a single MySQL server. It does not include login roles, mobile offline collection, real personal data, transport routing, procurement, or automated eligibility decisions. These boundaries keep the work appropriate for a second-year DBMS course and let the report concentrate on entity design, integrity, normalization, joins, and transaction behavior.

Chapter 2 defines the problem, objectives, constraints, and success measures. Chapter 3 reviews related systems and principles. Chapter 4 explains the schema and transaction design. Chapter 5 covers the implemented modules and screens. Chapter 6 records checked output and tests. Chapter 7 concludes with limitations and possible extensions.

Seven reporting questions guide the design: population by camp; needs still open; warehouse stock by item; distribution history by family; donor contributions; volunteer work; and referrals still awaiting completion. The answers are stored as SQL views so the same joins can be inspected independently of the browser interface. This makes the database itself the main project artifact.

2 PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem definition

Manual or separate spreadsheets can contain a camp roster, a needs register, a donation register, and an issue book. These records refer to the same families, items, volunteers, and warehouses but may use different spellings or identifiers. Duplicate entries and delayed updates can make a remaining quantity unreliable. The problem is to create one consistent data model that can answer operational questions while preventing invalid issue records.

The system must link each family to one current camp; each need to a family, item, relief round, quantity, urgency, and assessment time; each receipt to a donor, warehouse, and item; and each distribution to an assessed need, warehouse, and volunteer assignment. A stock movement must explain each balance change. These links are represented by foreign keys; row-level validity is supported by unique and check constraints [10], [11].

2.2 Objectives

Design a relational schema that represents the Project 60 entities and supports reports without repeating descriptive data unnecessarily.

Implement MySQL keys, constraints, views, and transactions that protect stock and need quantities.

Build a small Python interface that demonstrates registration, assessment, donation, assignment, issue, referral, search, and reports.

Show checked example output and tests for normal and invalid operations.

2.3 Constraints and success measures

The project is a classroom demonstration with fictional records. A local MySQL instance and Python environment are available, but no organization has supplied real users or field requirements. The workflow assumes a coordinator enters accurate assessments and that an operator records every receipt and issue through the application. It also assumes item quantities are whole numbers in a named unit, such as kits or blankets.

Requirement	Evidence in project	Success check
Unique family	Registration code unique key	Repeat code rejected
One round need	Family + item + round key	Duplicate assessment rejected
No excess issue	Locked need and stock rows	Over-need issue rejected
Traceable stock	Receipt/issue movement rows	Ledger reconciles
Useful reports	Seven SQL views	Checked query rows

Table 1. Requirements mapped to implemented evidence.

Success is defined as an understandable schema and a repeatable demonstration, not a deployment metric. The verified three-kit issue changes a family's outstanding water need from five to two and warehouse stock from twenty to seventeen. A test suite of eight MySQL integration cases passes on a fresh test database [18].

3 LITERATURE REVIEW AND BACKGROUND

3.1 Relational model and humanitarian information

Codd's relational model separated logical data relationships from physical storage and provided the foundation for representing records as relations [2]. This matters in the present project because family, camp, item, and distribution facts have different lifecycles. A person can be registered without a distribution, and a donation can exist without any specific family need. Keeping them in separate relations avoids repeating all details whenever one event changes.

Sangiamkul and van Hillegersberg reviewed information systems in humanitarian logistics and identified the value of information technology for relief operations while noting differing contexts and research methods [3]. Their work supports choosing a focused information-flow problem for an educational system: the project does not attempt a universal disaster platform, but it links the few records needed to trace a local camp issue.

The Sphere Handbook describes humanitarian standards in water, food, shelter, and health [4]. The project's item categories are inspired by the idea that needs are not a single undifferentiated number. This report does not claim compliance with Sphere standards; quantity targets would require field-specific policy and professional review.

3.2 Existing disaster and registration systems

Sahana Eden is an open-source disaster-management platform with modules for facilities, volunteers, inventories, warehouses, and supply requests [5]. It demonstrates that the domain contains several connected processes. The student system adopts only the minimum data relationships needed for a local camp demo. Its smaller scope makes each key and rule visible during a viva, but it lacks the integration and operational breadth of Sahana.

IFRC GO collects and presents emergency information to connect needs with response and support decision making [6]. It works at a broader situational-awareness level. The present database instead focuses on local transaction records: a family need, warehouse balance, volunteer assignment, and issue event. The two approaches answer different questions, and the report does not treat its local tables as a replacement for a response-wide information platform.

UNHCR describes PRIMES as an ecosystem for registration, identity, case management, reporting, and interoperability [7]. This highlights the significance of stable identifiers and careful data handling. Our family registration code is a basic classroom identifier, not a verified identity system. No photographs, biometrics, or government identifiers are stored in the seed data.

3.3 Resource inventory and field data collection

FEMA's Resource Inventory System guidance stresses current counts and consistent resource descriptions for response planning [9]. It covers equipment, personnel, teams, facilities, and supplies. The project's stock table uses one row for each warehouse-item pair, while the movement ledger stores the receipt or issue that changed it. The design gives a simple way to reconcile the balance with the event history.

ODK provides online and offline form-based data collection [8]. That is relevant because field registration can occur under poor connectivity. The current Python application runs locally against MySQL and does not synchronize offline submissions. An offline form could be a later extension, but it would require careful conflict handling so duplicate registrations and stock issues remain controlled.

The reviewed sources collectively suggest that data quality, traceability, and reporting are central to a relief workflow. The present implementation uses familiar DBMS concepts to model those needs. Its contribution is a teachable end-to-end example, not a novel humanitarian standard or a large-scale platform.

3.4 Comparison and design takeaway

Source	Main focus	Lesson used here
Sahana Eden [5]	Broad disaster operations	Link camps, stock, volunteers
IFRC GO [6]	Emergency information	Present useful status reports
UNHCR PRIMES [7]	Registration and identity	Use stable record identifiers
FEMA RIS [9]	Resource inventory	Track current and historical stock
ODK [8]	Field forms	Keep data entry simple
This project	Local DBMS learning	Explain constraints and transactions

Table 2. Related systems and the focused classroom implementation.

The comparison shows why the prototype is deliberately small. Full disaster platforms handle coordination, interoperability, data protection, and different field conditions. A student project can show a narrower chain of facts with clear keys and rules: register a family, assess need, receive stock, assign a volunteer, issue supplies, and inspect reports. This supports the review criteria while leaving realistic limitations visible.

The remaining chapters describe the actual design and implementation. All reported values come from the repository's fictional seed and checked demonstration. External systems are background references only; the application does not connect to them.

4 METHODOLOGY AND SYSTEM DESIGN

4.1 Staged development

The work was organized around the three review stages specified for the PBL assignment. Review 1 recorded the problem, scope, ER diagram, and first relational mapping. Review 2 implemented the MySQL tables, sample data, views, normalization notes, and a command-line prototype. Review 3 connected a Python application, added screenshots and tests, and assembled the final report [1], [18].

This order exposed a useful correction. The first schema draft repeated item_id in distributions. When the need relationship was examined for normalization, it became clear that need_id already determines the item. The implemented distribution table therefore stores need_id and obtains the item from needs. A transaction then locks the matching stock row before the issue.

The design was checked against three kinds of questions: structure (can every fact be represented without ambiguity?), integrity (can invalid quantities and duplicates be rejected?), and reporting (can common camp questions be answered with joins or views?). The MySQL and Python files in the repository are the implementation evidence for each decision.

The first review's schema was not treated as final merely because a diagram existed. The second review compared functional dependencies and built actual DDL, then the third review exercised the design through form submissions and concurrent operations. This progression made it possible to correct a structural issue before building more screens around it.

4.2 Overall architecture

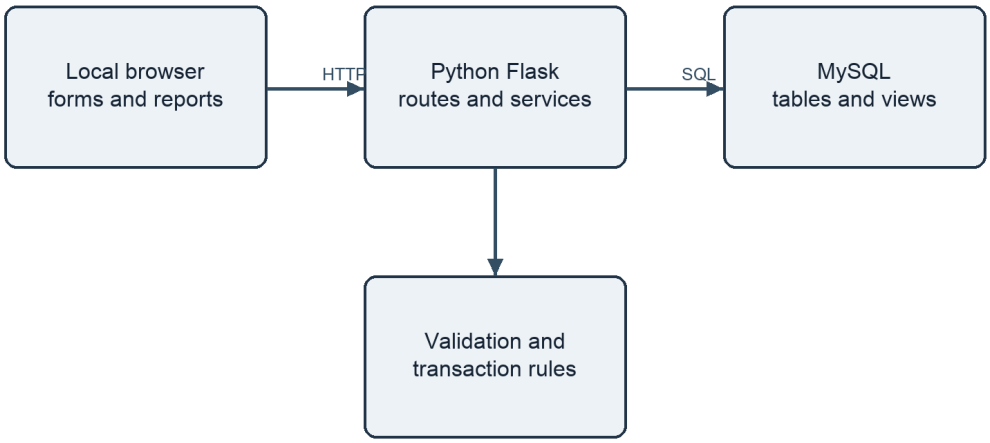


Figure 1. Three-layer architecture of the local demonstration.

The browser shows forms and reports. Flask routes read submitted fields and call Python service functions. Those functions run parameterized SQL against MySQL and return either a saved result or a validation message. The service layer owns the operations that must be atomic, including donation receipt and item distribution. Reporting views provide reusable joins and calculations. This is a local three-layer arrangement; it has no cloud component.

MySQL stores both reference records, such as relief items, and event records, such as donations and distributions. A current stock balance is stored for fast checking, while each stock movement records why the balance changed. The separation allows the interface to remain simple while the database retains traceable relationships.

4.3 Entity relationship design

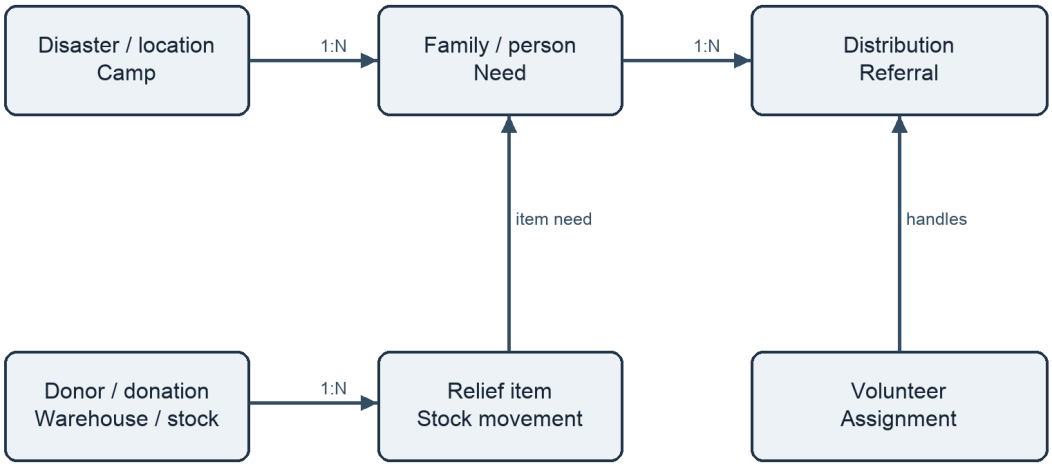


Figure 2. Core entity groups; detailed keys are defined in schema.sql.

Entity group	Primary identifiers	Important relationship
Camp registry	disaster_id, camp_id, family_id	Camp has many families
Needs	need_id, item_id	Family has many needs
Supply	donation_id, stock pair	Warehouse holds items
Distribution	distribution_id	Need may have partial issues
Staff	volunteer_id, assignment_id	Assignment links camp

Table 3. Main entity groups and their identifiers.

4.4 Normalization and dependencies

A single paper row combining camp, family, members, needs, donor, warehouse, and issue details has repeating groups. First Normal Form is achieved by storing one value per field and separate rows for members and item needs. Second Normal Form is straightforward because most tables use single-column surrogate keys. The stock table uses a composite key (warehouse_id, item_id), and quantity_on_hand depends on the complete pair.

For Third Normal Form, descriptions remain with the entity they describe. The item name and unit are stored in relief_items; the camp name is stored in camps; a distribution stores need_id, warehouse_id, assignment_id, quantity, token, and time. Reports join the tables to display descriptions. The dependency need_id → item_id is why the implemented distribution table does not repeat item_id.

Stock is a deliberate operational balance that duplicates a sum obtainable from movements. It remains useful for locking and fast issue checks. The exception is controlled by a transaction that writes both the balance and its event together, plus a reconciliation query that compares quantity_on_hand with the movement total. This is explained in the normalization notes [18].

For example, family_id determines camp_id and registration_code in families, while need_id determines family_id, item_id, relief_round, required quantity, and urgency in needs. A distribution does not store the registration code or item name; those values are reached by joins through need and family. If a camp name changes, only the camp row changes. This avoids a familiar update anomaly in a combined register.

4.5 Integrity rules and transaction method

Rule	Mechanism	Reason
Positive quantities	CHECK constraints	Reject zero or negative events
One need per round	UNIQUE composite key	Prevent duplicate assessment
Known parent rows	FOREIGN KEY	Avoid orphan events
No excess issue	Row locks + service check	Depends on earlier issues
No overlap	Volunteer lock + interval check	Depends on other assignments

Table 4. Representative database and service integrity rules.

Simple row rules are declared in SQL. Cross-row rules need a transaction because MySQL CHECK expressions cannot query other rows [11]. The service begins a READ COMMITTED transaction, locks the relevant need and stock rows with SELECT ... FOR UPDATE, checks remaining quantities, inserts the issue and matching movement, updates stock, then commits. If a check fails, it rolls back. MySQL documents locking reads for this read-then-update pattern [13].

4.6 Data flow and report design

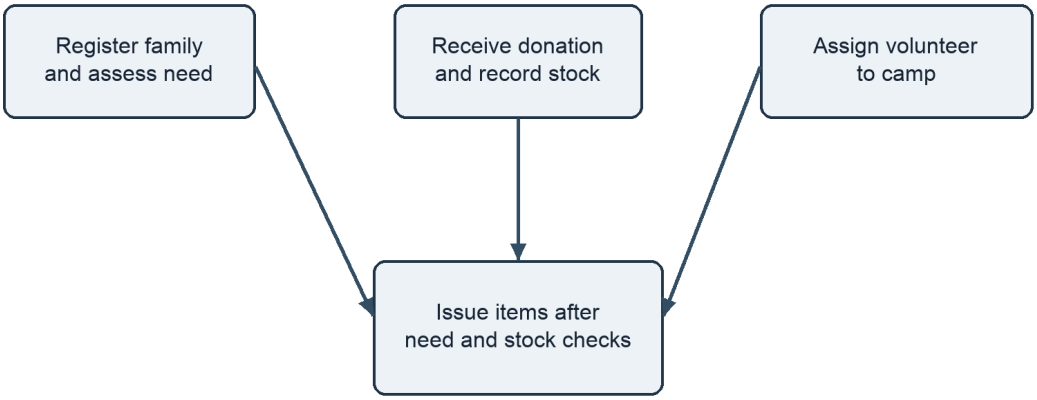


Figure 3. Inputs and validation converge on a recorded distribution.

The distribution path is reached only after a registered family has an assessed need, a warehouse has stock, and a volunteer has a valid camp assignment. The service checks that the assignment covers the issue time and camp. It uses a unique request token to reject replay of the same issue request. A similar receipt token protects donation entry.

Seven SQL views answer recurring questions: population by camp, unmet needs, stock by warehouse, distribution history, donor contributions, volunteer work, and open referrals. The report query file also demonstrates an aggregate, a nested query, and stock-ledger reconciliation. Views are useful here because a repeated join and calculation can be named once and queried as a relation [12].

5 IMPLEMENTATION

5.1 Modules and files

Module	Responsibility	Evidence
schema.sql	Tables, constraints, indexes, views	16 tables, 7 views
seed.sql	Fictional demo records	Repeatable example
reports.sql	Joins and report queries	Checked output
services.py	Validation and transactions	Atomic receipt and issue
web.py	Forms and reports	Local Flask interface
test_workflow.py	Integration checks	Eight passing cases

Table 5. Module responsibilities in the repository.

The repository keeps SQL, application code, tests, and review documentation in separate folders [18]. This makes it possible to show the schema independently of the interface. It also helps the professor inspect how a form action maps to a database operation.

The service functions receive plain values from the routes, validate them, and execute parameterized statements. Routes remain responsible for displaying forms and messages. The MySQL schema is responsible for primary and foreign keys, unique constraints, checks, indexes, and views. This separation is useful in a DBMS project because it makes the division between database enforcement and application enforcement explicit.

The command-line prototype in scripts/prototype.py provides an additional way to demonstrate the database without the browser. It can be used during Review 2 to show sample queries before the final interface is discussed. The same schema and fictional seed support both demonstrations.

5.2 Registration, assessment, and supply

A family is registered to a camp with a unique registration code. Member rows belong to the family through a foreign key. The family detail page shows the people and their assessed needs. A need row stores item, relief round, required quantity, urgency, and assessment time. The unique combination of family, item, and round prevents accidental duplicate entitlement records.

The donation workflow records a donor, warehouse, item, positive quantity, and receipt token. In one transaction it inserts a donation, updates or creates the corresponding warehouse-item stock balance, and writes a positive stock movement. The unique receipt token prevents a repeated form submission from silently doubling stock.

The item catalogue supports create, edit, and delete actions so the demonstration includes basic CRUD. The schema prevents deletion of an item when other records still reference it. This is preferable to silently losing an item name from past donation or distribution history.

```
INSERT INTO needs (family_id, item_id, relief_round,
                   quantity_required, urgency)
VALUES (%s, %s, %s, %s, %s);
-- Unique family/item/round is enforced by MySQL.
```

5.3 Volunteers, distribution, and referrals

A volunteer is assigned to a camp for a start and end time. The service rejects an interval that overlaps another assignment for the same volunteer. A distribution selects a need, warehouse, assignment, and quantity. Before posting, the service checks that the volunteer is assigned to the family's camp at issue time, that the need remains open, and that the warehouse has enough of the needed item.

The distribution record and a negative movement are written in the same transaction as the stock decrement. The remaining need is computed from assessed quantity minus the sum of distribution quantities. This allows partial issues without changing the original assessment. A referral records a person, destination, reason, date, and status; open referrals are exposed through a view.

The user interface presents validation errors in the form workflow. For example, a duplicate request token or quantity above remaining need produces a rejection rather than an incorrect balance. The transaction rollback means that a failed issue leaves both stock and movement history unchanged.

```
BEGIN;
SELECT ... FROM needs WHERE need_id = %s FOR UPDATE;
SELECT ... FROM stock WHERE warehouse_id = %s
  AND item_id = %s FOR UPDATE;
-- Check remaining need, stock and assignment.
-- Insert distribution; update stock; insert movement.
COMMIT;
```

The code sketch shows the order of checks; the exact statements are in app/services.py. The assignment row is also checked against the family's camp and the current time. A failure at any step raises an error and rolls the transaction back. The test suite includes a concurrent issue case to verify that two requests cannot jointly exceed the same need.

5.4 Fictional dataset and setup

Record type	Seed count	Purpose
Disaster	1	Flood scenario
Camps	2	Population grouping
Families / people	3 / 7	Registration and counts
Item needs	4	Urgency queue
Relief items	3	Water, food, blanket
Warehouse / volunteers	1 / 2	Supply and assignment
Referral	1	Open case report

Table 6. Fictional records supplied by seed.sql.

The setup creates a MySQL database, runs schema.sql, then seed.sql. The Python requirements install Flask and the MySQL connector. Connection settings are supplied through environment variables documented in .env.example. The app runs locally and reads or writes the same database used by the report queries [18].

The fictional records give a small enough dataset to verify every expected number by inspection. They are demonstrations, not a dataset drawn from real affected people.

5.5 Application overview

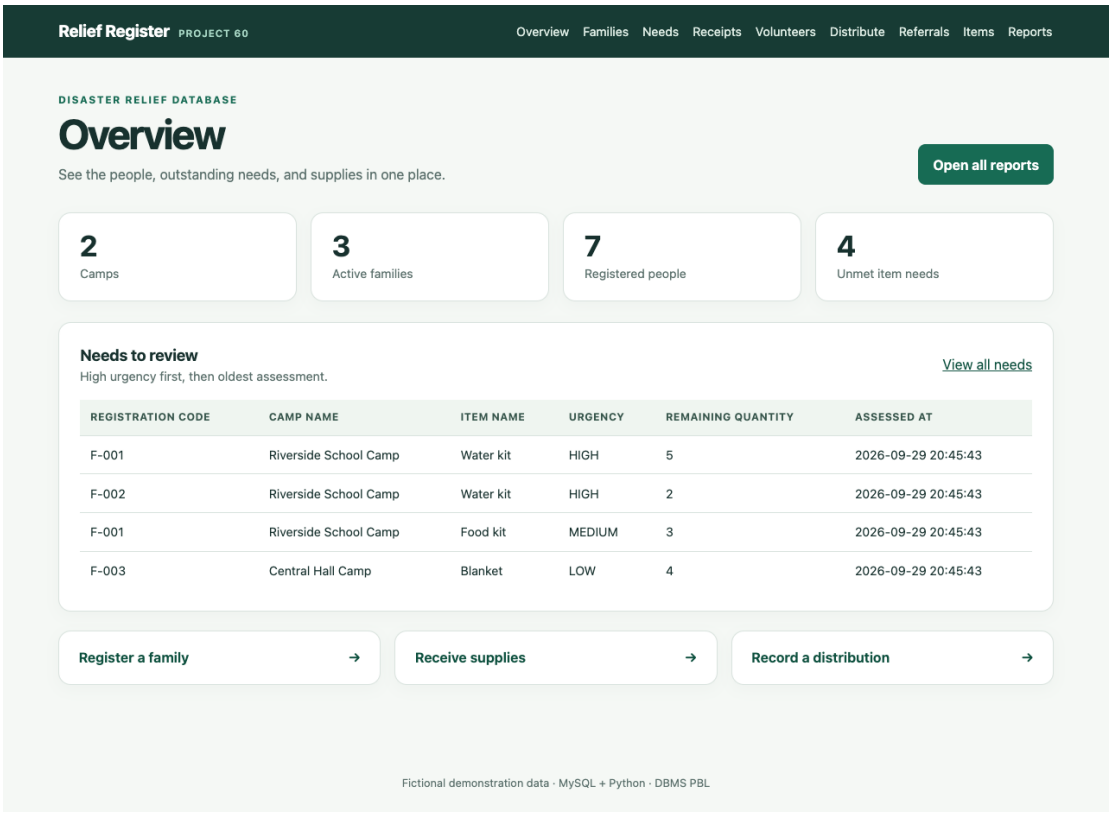


Figure 4. Local application overview with fictional camp and need data.

The overview brings the high-priority unmet-needs queue close to the camp and family counts. A user can move from the queue to registration, assessment, receipts, distributions, and reports. The queue orders urgency first and older assessments next; it does not automatically authorize a distribution.

5.6 Distribution and reports screens

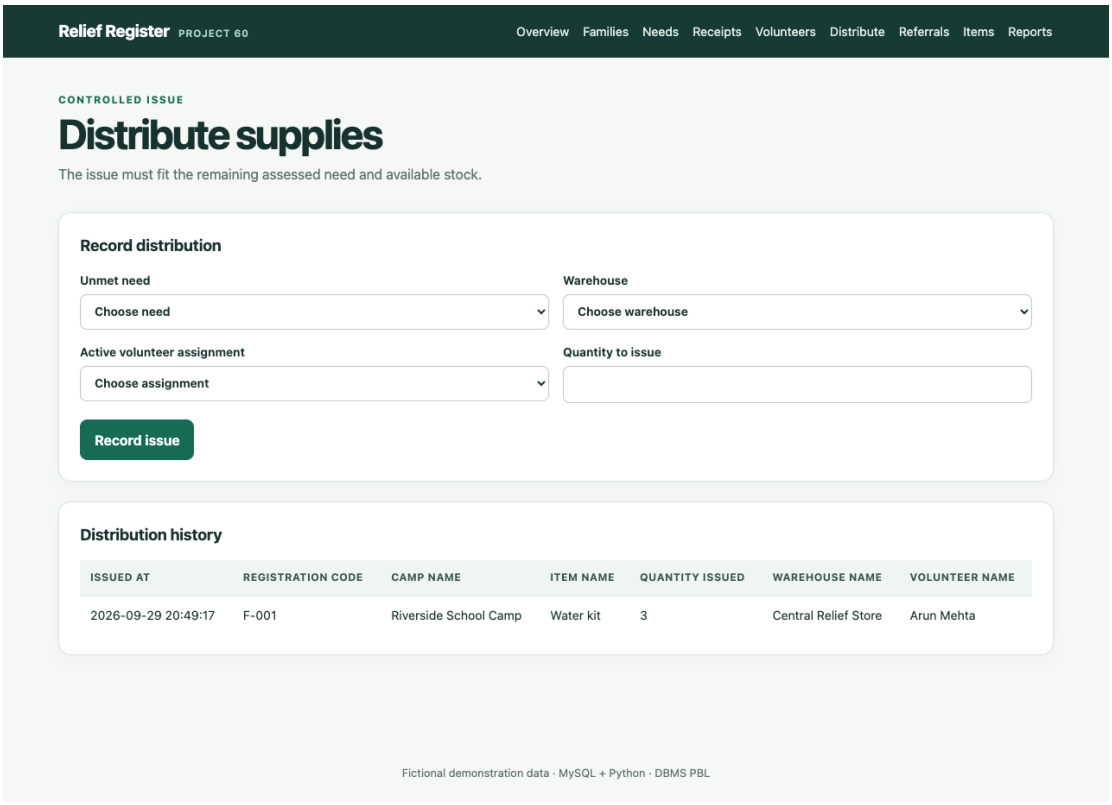


Figure 5. Distribution form and recorded issue history.

The issue form requires a recorded need, warehouse, volunteer assignment, and quantity. After submission, the history identifies the family registration code, item, warehouse, volunteer, and quantity. The reports page additionally shows current stock and the remaining need, so the demonstrated change can be checked from two viewpoints.

6 RESULTS AND DISCUSSION

6.1 Checked query output

Query or measure	Verified seed result
Central Hall Camp	1 family, 1 person
Riverside School Camp	2 families, 6 people
F-001 Water kit	Required 5; initially issued 0
Water kit stock	20 before distribution
F-001 after issue of 3	Remaining need 2
Water kit stock after issue	17
Stock reconciliation	No mismatched rows

Table 7. Values checked against the fictional MySQL seed and demonstration.

The population view joins camps, families, and people. The unmet-needs view subtracts issued quantity from assessed quantity, and excludes fully served needs. The nested query in reports.sql finds F-001's five water kits as above the average assessed water-kit quantity. These examples demonstrate joins, grouping, a subquery, and a reusable view rather than separate hard-coded totals.

The two camps also show why reports should count carefully. The family count uses distinct family identifiers, while the person count counts member rows. Joining both without the correct grouping could multiply family counts. The view fixes the grouping by camp and keeps the meaning of each number visible in its column name.

6.2 Interpretation of the demonstration

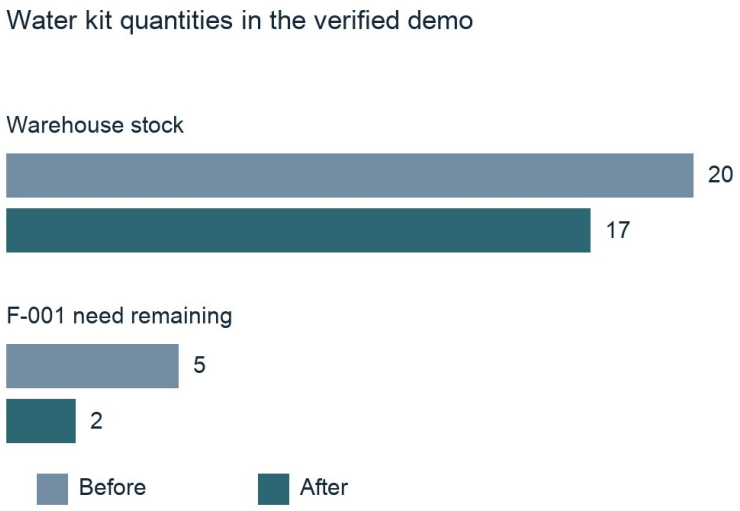


Figure 6. Verified quantities before and after one three-kit issue.

The two bars show quantities, not response speed. A successful three-kit issue reduced warehouse stock by three and the family's remaining water need by three. The movement ledger records an ISSUE of -3. A repeated request token was rejected, and a further issue of three was rejected because only two kits remained. The reconciliation query returned no mismatch after the issue.

This result supports the main claim of the project: a connected relational workflow can keep need and stock views consistent for this small fictional case. It does not establish performance under a large workload, reliability without connectivity, or suitability for real disaster operations.

6.3 Testing and objective review

Test area	Observed outcome
Concurrent issues	Combined issues cannot exceed need
Duplicate / excess issue	Rejected
Duplicate assessment	Rejected
Overlapping volunteer time	Rejected
Receipt and movement	Stock and ledger increase together
Ledger reconciliation	No mismatched stock rows
Web family workflow	Registration and need saved
Web pages and item CRUD	Pages and create/edit/delete checked

Table 8. Eight passing MySQL integration test areas.

The tests were run on a fresh disposable MySQL database and all eight passed [18]. The tests support correctness for the covered paths, especially the cross-row distribution rule. They do not measure throughput or prove that every possible failure has been handled. The objectives in Chapter 2 are met for the local demonstration: the schema exists, the app connects to it, reports reproduce checked results, and invalid actions are rejected.

A fresh database matters because a leftover distribution would change the expected remaining need and stock values. The test setup loads the schema and fictional seed, applies each operation, and inspects both the application result and the affected database records. The reconciliation check compares each stock balance with the sum of receipt and issue movements, which is stronger than checking only the number shown on a web page.

7 CONCLUSION AND FUTURE SCOPE

The project produced a MySQL database and a Python interface for a small disaster relief camp workflow. Sixteen tables represent the assignment entities and the added donor and stock-movement history; seven views answer recurring questions. The design separates facts so that names and descriptions are not repeated in event rows, while constraints and transactions protect quantities and relationships. The fictional demo and eight integration tests show a traceable three-kit distribution with matching changes in need and stock.

The main learning outcome is that basic DBMS techniques work together: an ER model guides table design; functional dependencies guide normalization; keys preserve relationships; SQL views support reports; and transactions protect multi-row actions. The explainable urgency-and-time queue shows how a useful feature can be built from ordinary SQL rather than a complex scoring method.

Future work could add: (1) authenticated roles for data entry and approval; (2) offline field capture with safe synchronization; (3) family transfer history between camps; (4) audited corrections and reversals for mistaken issues; and (5) larger test datasets and measured query performance. Each change would require additional operational requirements and privacy review before real use.

REFERENCES

[1] Woxsen University, "DBMS PBL 70 Unique Project Statements," Project 60 assignment document, 2026.

[2] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970. doi: 10.1145/362384.362685.

[3] E. Sangiamkul and J. van Hillegersberg, "Research Directions in Information Systems for Humanitarian Logistics," Proc. ISCRAM, Lisbon, 2011. Available: https://research.utwente.nl/en/publications/research-directions-in-information-systems-for-humanitarian-logis/

[4] Sphere Association, The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response, 4th ed., 2018. Available: https://spherestandards.org/handbook/

[5] Sahana Software Foundation, "Sahana EDEN: Open Source Disaster Management." Available: https://new.sahanafoundation.org/

[6] International Federation of Red Cross and Red Crescent Societies, "IFRC GO." Available: https://go.ifrc.org/

[7] UNHCR, "Registration Tools: PRIMES," Guidance on Registration and Identity Management. Available: https://www.unhcr.org/registration-guidance/chapter3/registration-tools/

[8] ODK, "Getting Started With ODK." Available: https://docs.getodk.org/getting-started/

[9] Federal Emergency Management Agency, "Resource Inventorying," National Resource Hub. Available: https://preptoolkit.fema.gov/web/national-resource-hub/resourceinventorying

REFERENCES CONTINUED

[10] Oracle, "FOREIGN KEY Constraints," MySQL 8.4 Reference Manual. Available: https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html

[11] Oracle, "CHECK Constraints," MySQL 8.4 Reference Manual. Available: https://dev.mysql.com/doc/refman/8.4/en/create-table-check-constraints.html

[12] Oracle, "CREATE VIEW Statement," MySQL 8.4 Reference Manual. Available: https://dev.mysql.com/doc/refman/8.4/en/create-view.html

[13] Oracle, "Locking Reads," MySQL 8.4 Reference Manual. Available: https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

[14] Oracle, "Introduction to InnoDB," MySQL 8.4 Reference Manual. Available: https://dev.mysql.com/doc/refman/8.4/en/innodb-introduction.html

[15] Pallets, "Flask Quickstart." Available: https://flask.palletsprojects.com/en/stable/quickstart/

[16] Pallets, "Testing Flask Applications." Available: https://flask.palletsprojects.com/en/stable/testing/

[17] Python Software Foundation, "unittest — Unit testing framework." Available: https://docs.python.org/3/library/unittest.html

[18] M SHREEVENKAT SATHVIK, "Disaster Relief Resource and Camp Management System," project repository, 2026. Available: https://github.com/sathvikmallepalli-lgtm/disaster-relief-dbms-pbl